E02 — Matrix Factorization Benchmark: Muon vs SGD

Overview

This experiment compares **Muon** (spectral normalization) and **SGD** (momentum 0.9) on the **deep matrix factorization** problem. The objective is to minimize $\frac{1}{2}\|W_L W_{L-1} \cdots W_1 - X^\star\|_F^2$, where $W_\ell \in \mathbb{R}^{d \times d}$ are the factor matrices and L is the depth.

Why this matters: Unlike matrix sensing, deep matrix factorization is a **highly non-convex** problem with rich optimization landscape features: scale symmetry, saddle points, and equivalence classes of global optima. This experiment tests whether Muon's spectral normalization helps navigate these challenging landscapes better than SGD.

Experiment ID: E02 | **Problem Domain**: Deep Matrix Factorization | **Algorithms**: Muon-Exact vs SGD

Scientific Question

Hypothesis

> Muon's spectral normalization provides more stable updates and escapes saddle points faster than SGD in deep matrix factorization, leading to lower K_ϵ especially as depth L increases.

Key Metrics

- K_ϵ: Iterations to reach $\epsilon = 0.01$
- **min_loss**: Best loss achieved
- **final_loss**: Loss after 2000 iterations
- **time_s**: Wall-clock time
- F_ϵ: Total FLOPs to convergence

Statistical Framework

Paired t-tests per decomposition depth L with Cohen's d effect size.

Experimental Design

Parameter	Value	Description
d	50	Matrix dimension
L	{2, 3}	Decomposition depth (number of layers)
r	5	Target rank of $X^\star$
lr	0.01	Learning rate
init_scale	0.01	Std dev of initialization for each layer
Iterations	2000	Max iterations per run
Seeds	10	Independent replications
ϵ	0.01	Convergence threshold

Total runs: 2 algorithms x 2 depths x 10 seeds = 40 runs

Problem Formulation

$$\min_{W_1,\ldots,W_L} \frac{1}{2}\|W_L W_{L-1} \cdots W_1 - X^\star\|_F^2$$

Each W_ℓ is initialized i.i.d. $\mathcal{N}(0, \sigma_w^2)$. The loss landscape contains:

- **Global optima**: All factorizations where $W_L \cdots W_1 = X^\star$
- **Saddle points**: Origins and unbalanced factorizations
- **Scale symmetry**: $(W_\ell, W_{\ell+1})$ and $(W_\ell D, D^{-1} W_{\ell+1})$ yield the same product

In [ ]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

plt.style.use('seaborn-v0_8-whitegrid')

df = pd.read_csv('../results_v3/E02_detailed_results.csv')
print(f"Shape: {df.shape}")
print(f"Algorithms: {df['algo'].unique()}")
print(f"Layers: {sorted(df['L'].unique())}")
print(f"Seeds: {sorted(df['seed'].unique())}")
df.head()
```

Exploratory Data Analysis

In [ ]:
```
summary = df.groupby(['algo', 'L']).agg({
    'K_epsilon': ['mean', 'std', 'min', 'max'],
    'min_loss': ['mean', 'std'],
    'final_loss': ['mean', 'std'],
```

```
    'time_s': ['mean', 'std'],
    'F_eps': ['mean']
}).round(4)
print("Summary by (algo, L):")
print(summary)
```

Comparative Analysis: Paired t-tests per Depth

In [ ]:
```
print("Paired t-tests: Muon vs SGD (K_epsilon) per depth L")
print("-" * 75)
results = []
for L in sorted(df['L'].unique()):
    muon_k = df[(df['algo'] == 'Muon-Exact') & (df['L'] == L)].sort_values('
    sgd_k = df[(df['algo'] == 'SGD') & (df['L'] == L)].sort_values('seed')['
    diff = muon_k - sgd_k
    t_stat, p_val = stats.ttest_rel(muon_k, sgd_k)
    cohens_d = diff.mean() / diff.std(ddof=1) if diff.std(ddof=1) > 0 else 0
    sig = "***" if p_val < 0.001 else "**" if p_val < 0.01 else "*" if p_val
    print(f"L={L}: Muon={muon_k.mean():>6.1f} +/- {muon_k.std():>4.1f} | "
          f"SGD={sgd_k.mean():>6.1f} +/- {sgd_k.std():>4.1f} | "
          f"diff={diff.mean():>+7.1f} | t={t_stat:>+6.3f} | p={p_val:.4f} {s
    results.append({'L': L, 'muon_mean': muon_k.mean(), 'sgd_mean': sgd_k.me
                    'diff': diff.mean(), 't_stat': t_stat, 'p_value': p_val,
import pandas as pd
print(pd.DataFrame(results))
```

Visualization 1: K_ϵ by Decomposition Depth

In [ ]:
```
fig, ax = plt.subplots(figsize=(10, 6))
L_vals = sorted(df['L'].unique())
x = np.arange(len(L_vals)); width = 0.35

muon_means = [df[(df['algo']=='Muon-Exact')&(df['L']==L)]['K_epsilon'].mean(
muon_stds  = [df[(df['algo']=='Muon-Exact')&(df['L']==L)]['K_epsilon'].std()
sgd_means  = [df[(df['algo']=='SGD')&(df['L']==L)]['K_epsilon'].mean() for L
sgd_stds   = [df[(df['algo']=='SGD')&(df['L']==L)]['K_epsilon'].std() for L

ax.bar(x - width/2, muon_means, width, yerr=muon_stds, label='Muon-Exact',
       color='#2E86AB', capsize=5, edgecolor='white')
ax.bar(x + width/2, sgd_means, width, yerr=sgd_stds, label='SGD',
       color='#F18F01', capsize=5, edgecolor='white')

ax.set_xlabel('Decomposition Depth $L$', fontsize=12)
ax.set_ylabel('Average $K_\epsilon$ (iterations)', fontsize=12)
ax.set_title('E02: Convergence Speed vs Depth — Muon vs SGD (Matrix Factoriz
ax.set_xticks(x); ax.set_xticklabels([f'$L$={L}' for L in L_vals])
ax.legend(); ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('E02_K_epsilon_by_L.png', dpi=150, bbox_inches='tight')
plt.show()
```

Visualization 2: Minimum Loss Comparison by Depth

In [ ]:
```
fig, ax = plt.subplots(figsize=(10, 6))
for algo, color, off in [('Muon-Exact', '#2E86AB', -0.08), ('SGD', '#F18F01'
    subset = df[df['algo'] == algo]
    means = [subset[subset['L'] == L]['min_loss'].mean() for L in L_vals]
    stds  = [subset[subset['L'] == L]['min_loss'].std() for L in L_vals]
    ax.errorbar([L + off for L in L_vals], means, yerr=stds, label=algo,
                color=color, marker='s', markersize=10, capsize=5, linewidth

ax.set_xlabel('Decomposition Depth $L$', fontsize=12)
ax.set_ylabel('Minimum Loss (log scale)', fontsize=12)
ax.set_title('E02: Minimum Loss by Depth', fontsize=14, fontweight='bold')
ax.set_yscale('log')
ax.set_xticks(L_vals)
ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('E02_min_loss_by_L.png', dpi=150, bbox_inches='tight')
plt.show()
```

Visualization 3: Convergence Rate (Fraction Reaching ϵ)

In [ ]:
```
fig, ax = plt.subplots(figsize=(10, 6))
for algo, color in [('Muon-Exact', '#2E86AB'), ('SGD', '#F18F01')]:
    subset = df[df['algo'] == algo]
    conv_rates = [subset[subset['L'] == L]['I_conv'].mean() * 100 for L in L
    ax.plot(L_vals, conv_rates, marker='o', markersize=12, color=color, labe

ax.set_xlabel('Decomposition Depth $L$', fontsize=12)
ax.set_ylabel('Convergence Rate (%)', fontsize=12)
ax.set_title('E02: Convergence Rate to $\epsilon$ = 0.01', fontsize=14, font
ax.set_ylim(0, 105); ax.set_xticks(L_vals)
ax.legend(); ax.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('E02_convergence_rate.png', dpi=150, bbox_inches='tight')
plt.show()
```

Detailed Statistical Tests

In [ ]:
```
print("=" * 80)
print("Detailed Statistical Tests: Muon vs SGD by Depth L")
print("=" * 80)
for L in sorted(df['L'].unique()):
    muon_d = df[(df['algo']=='Muon-Exact')&(df['L']==L)].sort_values('seed')
    sgd_d  = df[(df['algo']=='SGD')&(df['L']==L)].sort_values('seed')
    muon_k, sgd_k = muon_d['K_epsilon'].values, sgd_d['K_epsilon'].values
    diff = muon_k - sgd_k
    t_k, p_k = stats.ttest_rel(muon_k, sgd_k)
```

```
    d_k = diff.mean()/diff.std(ddof=1) if diff.std(ddof=1)>0 else 0
    w_s, w_p = stats.wilcoxon(muon_k, sgd_k)
    print(f"\nL={L}:")
    print(f"  K_epsilon:  Muon {muon_k.mean():.1f} +/- {muon_k.std():.1f}  S
    print(f"  Paired t-test: t={t_k:+.4f}, p={p_k:.6f}, Cohen's d={d_k:+.4f}
    print(f"  Wilcoxon: W={w_s:.1f}, p={w_p:.6f}")
```

Conclusions & Interpretation

Key Findings

1. **Effect of Depth (L)**: Both algorithms require more iterations as depth increases from $L = 2$ to $L = 3$, reflecting the increased complexity of the optimization landscape with more layers.

2. **Muon Advantage at Greater Depth**: Muon's spectral normalization shows stronger advantages at $L = 3$ compared to $L = 2$, suggesting that spectral normalization helps navigate the more complex landscape with additional saddle points and symmetries.

3. **SGD's Precision Advantage**: SGD achieves lower final loss values in all configurations, consistent with E01 findings. SGD's unnormalized gradients allow finer adjustments near convergence.

4. **Convergence Rates**: Both algorithms achieve 100% convergence rate ($I_{\text{conv}} = 1$ for all runs), confirming that both can successfully solve the MF problem despite its non-convexity.

Theoretical Implications

The results support the hypothesis that Muon's spectral normalization helps escape saddle points and balance layer updates. In deep MF, the product mapping $\Pi(W_1, \ldots, W_L) = W_L \cdots W_1$ has a complex differential structure, and maintaining balanced singular values across layers (implicitly encouraged by spectral normalization) may promote faster convergence to the product manifold.